



Verified computation of real powers of matrices

Shinya Miyajima

Faculty of Science and Engineering, Iwate University, 020-8551, Japan



ARTICLE INFO

Article history:

Received 18 March 2020

Received in revised form 16 January 2021

MSC:

15A16

65F60

65G20

Keywords:

Matrix real power

Verified block diagonalization

Verified numerical computation

ABSTRACT

Two numerical algorithms based on verified block diagonalization (VBD) are proposed to compute interval matrices containing the matrix real powers. The first algorithm uses VBD based on numerical spectral decomposition involving cubic complexity if the power exponent's absolute value is not too large. In contrast, the second algorithm adopts VBD based on numerical Jordan decomposition involving quartic complexity, which is applicable even for defective matrices. Numerical results show the effectiveness of the algorithms.

© 2021 Elsevier B.V. All rights reserved.

1. Introduction

Let $\alpha \in \mathbb{R}$ and $A \in \mathbb{C}^{n \times n}$ have no non-positive real eigenvalues. In this paper, we are concerned with the accuracy of numerically computed result for A^α . The power A^α is uniquely defined as follows.

Definition 1 (Higham and Lin [1, Definition 1.1]). Let $\alpha \in \mathbb{R}$, and $A \in \mathbb{C}^{n \times n}$ have no non-positive real eigenvalues. Then, $A^\alpha = e^{\alpha \log(A)}$, where $\log(A)$ is the principal logarithm of A [2, Theorem 1.31].

Remark 1. The matrix A is nonsingular because A has no non-positive real eigenvalues. In other words, this paper does not treat the case where A has zero eigenvalues. Hence, Definition 1 is simpler than [1, Definition 1.1].

If $\alpha = 1/d$, d being a positive integer, then A^α reduces to the principal d th root of A [2, Theorem 7.2]. If A has no non-positive real eigenvalues and $\alpha \in (0, 1)$, then the following representation is obtained [2, pp. 174, 187]:

$$A^\alpha = \frac{\sin(\alpha\pi)}{\alpha\pi} A \int_0^\infty (t^{1/\alpha} I_n + A)^{-1} dt, \quad (1)$$

where I_n denotes the $n \times n$ identity matrix.

The computation of A^α has many applications, such as Markov chain models in healthcare and finance [3,4], fractional partial differential equations [5], discrete representations of norms corresponding to finite element discretization of fractional Sobolev spaces [6], and computation of geodesic midpoints in neural networks [7]. Efficient numerical algorithms for computing A^α are proposed in previous studies [1,8,9].

In this paper, the problem of verified computations for A^α , specifically, numerically computing interval matrices containing A^α , is addressed. There are many well-established verification algorithms for matrix functions (see [10–20] for instance). To the best of the author's knowledge, a verification algorithm designed specifically for A^α is not yet

E-mail address: miyajima@iwate-u.ac.jp.

studied. If α is rational, then α can be written as $\alpha = c/d$, where $c, d \in \mathbb{Z}$ and $d > 0$, so that $A^\alpha = (A^{1/d})^c$. Therefore, the verification algorithm for the matrix principal d th root [15] is applicable. However, this approach cannot be adopted when α is irrational (see Section 5, in which the case where α is irrational is treated). One possible method is to use the VERSOFT [21] routine `vermatfun`, which can be applied not only to matrix real power but also to other matrix functions and can compute the interval matrices by enclosing all the eigenvalues and eigenvectors of A via the INTLAB [22] routine `verifyeig`. The routine `vermatfun` fails when A is defective or close to defective, and requires $\mathcal{O}(n^4)$ operations. Because $A^\alpha = e^{\alpha \log(A)}$, interval matrices can be computed by combining the algorithm for enclosing the matrix exponentials [10,11,16] and principal logarithms [17], giving interval matrices whose radii are not small (Section 5). This is because an interval matrix containing $\alpha \log(A)$ is computed in the first stage, and its radius is enlarged in the process of enclosing the matrix exponential.

This paper aims to propose two verification algorithms for A^α . Algorithms for enclosing all eigenvalues and bases of invariant subspaces of A are presented in [23]. Interval matrices containing blocks whose spectra are included in that of A are the byproducts obtained from these algorithms. The spectrum of A is also contained in the union of the spectra of the blocks. In [19], interval matrices containing the bases and blocks are the result of verified block diagonalization (VBD), which establishes a framework for enclosing matrix functions. On the basis of this framework, verification algorithms are developed. Computable error bounds, which are the main contributions of this paper, are derived to obtain enclosures for the blocks' real power. Here, the word "computable" means that a rigorous upper bound taking all possible errors into account can be numerically obtained. The first algorithm uses VBD based on numerical spectral decomposition (NSD) involving $\mathcal{O}(n^3)$ operations if $|\alpha|$ is not too large. The second algorithm adopts VBD based on numerical Jordan decomposition (NJD) requiring $\mathcal{O}(n^4)$ operations, which is applicable even when A is defective. On the basis of checking the nonsingularity of a matrix, a theory for verifying that A has no non-positive real eigenvalues is presented. With this theory, the algorithms do not assume but prove that A has no non-positive real eigenvalues. One may consider that the proposed algorithms are direct applications of the framework in [19]. However, verifying that A has no non-positive real eigenvalues based on checking nonsingularity and deriving the computable perturbation bounds for the matrix real powers are novel contributions and not obvious. Moreover, the perturbation bounds in this paper are componentwise, whereas those in [19] are normwise.

The rest of this paper is organized as follows. Section 2 introduces notation and theoretical results used in this paper. Sections 3 and 4 propose the first and second algorithms, respectively. Section 5 reports numerical results. Section 6 summarizes the results in this paper and highlights a possible extension and future work.

2. Preliminaries

For $M \in \mathbb{C}^{n \times n}$, let M_{ij} , M_j , $\kappa(M)$, $\rho(M)$, and $\sigma(M)$ be the (i, j) element, j th column, two-norm condition number, spectral radius, and spectrum of M , respectively; moreover, $|M| := (|M_{ij}|)$. For $v \in \mathbb{C}^n$, denote the i th element of v by v_i . For $M, N \in \mathbb{R}^{m \times n}$, the inequality $M \leq N$ means $M_{ij} \leq N_{ij}$, $\forall i, j$. For $v, w \in \mathbb{C}^n$ with $\|w\|_\infty < 1$, define $\|v\|_w := \max_i(|v_i|/(1 - |w_i|))$. Let $\mathbb{R}_- := (-\infty, 0]$, and $\mathbb{R}_+^{m \times n} := \{M \in \mathbb{R}^{m \times n} : M \geq 0\}$. Also, let $\mathbb{IC}$ and $\mathbb{IC}^{m \times n}$ be the sets of all complex interval scalars and $m \times n$ matrices, respectively. In the Gaussian plane, a complex interval scalar geometrically means a disk; thus, circular interval arithmetic is used (see [24, Chapter 5] for instance). For $C \in \mathbb{C}^{m \times n}$ and $R \in \mathbb{R}_+^{m \times n}$, let $\langle C, R \rangle$ denote the interval matrix whose midpoint and radius are C and R , respectively. Suppose any matrices contained in $\mathbf{M} \in \mathbb{IC}^{n \times n}$ are nonsingular. Then, $\mathbf{M}^{-1}$ denotes an interval matrix, enclosing $\{M^{-1} : M \in \mathbf{M}\}$. Expressions containing intervals mean results of interval arithmetic. Denote the set of all real closed interval scalars by $\mathbb{IR}$. For $\mathbf{a} \in \mathbb{IR}$, let $\inf(\mathbf{a})$, $\sup(\mathbf{a})$, $\text{mid}(\mathbf{a})$, and $\text{diam}(\mathbf{a})$ be the infimum, supremum, midpoint, and diameter of $\mathbf{a}$, respectively. For $\alpha \in \mathbb{R}$, let $\lfloor \alpha \rfloor$ denote the floor function. Let I_n be the $n \times n$ identity matrix, $\mathbb{1}_n := [1, \dots, 1]^T \in \mathbb{R}^n$, and

$$N_j := \begin{bmatrix} 0 & 1 & & \\ & 0 & \ddots & \\ & & \ddots & 1 \\ & & & 0 \end{bmatrix} \in \mathbb{R}^{n_j \times n_j}.$$

In Sections 3 and 4, the following properties of matrix functions are used.

Lemma 1 (See for Instance Higham [2, Theorem 1.13]). Let $A, X \in \mathbb{C}^{n \times n}$, and $f : \mathbb{C} \rightarrow \mathbb{C}$ be defined on $\sigma(A)$. Then,

- (a) if X is nonsingular, then $f(XAX^{-1}) = Xf(A)X^{-1}$;
- (b) if $A = \text{diag}(A_1, \dots, A_p)$ is block diagonal, then $f(A) = \text{diag}(f(A_1), \dots, f(A_p))$.

Lemma 2 is presented for verifying $\sigma(M) \cap \mathbb{R}_- = \emptyset$, where $M \in \mathbb{C}^{n \times n}$. The proof is omitted because it is obvious.

Lemma 2. Let $t \in [0, \infty)$ and $M \in \mathbb{C}^{n \times n}$. Then, $\sigma(M) \cap \mathbb{R}_- = \emptyset$ if and only if $tI_n + M$ is nonsingular for any t .

In the proofs of Theorems 1 and 2, upper bounds for $|M^{-1}|$ will be estimated, where $M \in \mathbb{C}^{n \times n}$ is nonsingular. To achieve the estimation, the author refers to Lemma 3 and presents Corollaries 1 and 2.

Lemma 3 (Miyajima [23]). Let $S, T \in \mathbb{C}^{n \times n}$ be given, and $s := |S|_{\mathbb{I}_n}$. Assume $\|s\|_{\infty} < 1$, and define $w := [\|T_{:1}\|_s, \dots, \|T_{:n}\|_s]^T$. Then, $I_n - S$ is nonsingular, and $|(I_n - S)^{-1}|T| \leq |T| + sw^T$.

Corollary 1. Let S, T , and s be as in Lemma 3. Suppose T is nonsingular, and define $t := |T^{-1}|_s$. Assume $\|t\|_{\infty} < 1$, and let $w := [\|T_{:1}^{-1}\|_t, \dots, \|T_{:n}^{-1}\|_t]^T$. Then, $T + S$ is nonsingular, and $|(T + S)^{-1}| \leq |T^{-1}|(I_n + sw^T)$.

Proof. Let $\widehat{t} := |-T^{-1}S|_{\mathbb{I}_n}$ and $\widehat{w} := [\|T_{:1}^{-1}\|_{\widehat{t}}, \dots, \|T_{:n}^{-1}\|_{\widehat{t}}]^T$. Then, $\widehat{t} \leq t$ and $\|\widehat{t}\|_{\infty} \leq \|t\|_{\infty} < 1$ are obtained, so that $\widehat{w} \leq w$. The inequalities, $T + S = T(I_n - (-T^{-1}S))$, and Lemma 3 applied to $S := -T^{-1}S$ and $T := T^{-1}$ prove that $T + S$ is nonsingular, and

$$|(T + S)^{-1}| \leq |(I_n - (-T^{-1}S))^{-1}| |T^{-1}| \leq |T^{-1}| + \widehat{t}\widehat{w}^T \leq |T^{-1}|(I_n + sw^T). \quad \square$$

Corollary 2. Let S and s be as in Lemma 3, and $\beta \in \mathbb{C} \setminus \{0\}$. Assume $\|s\|_{\infty} < |\beta|$, and define $w := [1/(|\beta| - s_1), \dots, 1/(|\beta| - s_n)]^T$. Then, $\beta I_n + S$ is nonsingular, and $|(\beta I_n + S)^{-1}| \leq (1/|\beta|)(I_n + sw^T)$.

Proof. Corollary 1 applied to $T := \beta I_n$ shows the result. $\square$

In the proofs of Theorems 1 and 2, it is necessary to estimate upper bounds for $\int_0^{\infty} t^{1/\alpha} / |t^{1/\alpha} + \widehat{\lambda}_j|^k dt$, $k = 2, 3, \dots$, where $\widehat{\lambda}_j \in \mathbb{C}$, and $\alpha \in (0, 1)$. For this purpose, Lemma 4 and Corollary 3 are presented.

Lemma 4. Let $\alpha \in (0, 1)$, $\widehat{\lambda}_j \in \mathbb{C}$, and $\mu_j := \max(0, \operatorname{Re}(\widehat{\lambda}_j)) + \operatorname{Im}(\widehat{\lambda}_j)\sqrt{-1}$. Suppose $|\mu_j| > 0$, and define $\varphi_k : (0, \infty) \rightarrow (0, \infty)$, $k = 2, 3, \dots$ by

$$\varphi_k(x) := \frac{\alpha x^{1/\alpha+1}}{(\alpha+1)|\mu_j|^k} + \frac{\alpha |\widehat{\lambda}_j|^k x^{1-(k-1)/\alpha}}{(k-\alpha-1)|\mu_j|^k}.$$

It then follows that

$$\int_0^{\infty} \frac{t^{1/\alpha}}{|t^{1/\alpha} + \widehat{\lambda}_j|^k} dt \leq \varphi_k(x), \quad \forall x \in (0, \infty). \quad (2)$$

Remark 2. Because $|t^{1/\alpha} + \widehat{\lambda}_j| \geq |\mu_j| > 0$, $\forall t \in [0, \infty)$, the integrands in Eq. (2) are well-defined.

Proof. Let $t \in [0, \infty)$. It follows that

$$|t^{1/\alpha} + \widehat{\lambda}_j|^2 - \frac{|\mu_j|^2}{|\widehat{\lambda}_j|^2} t^{2/\alpha} = \frac{|\widehat{\lambda}_j|^2 - |\mu_j|^2}{|\widehat{\lambda}_j|^2} t^{2/\alpha} + 2\operatorname{Re}(\widehat{\lambda}_j)t^{1/\alpha} + |\widehat{\lambda}_j|^2. \quad (3)$$

If $\operatorname{Re}(\widehat{\lambda}_j) \geq 0$, then $|\widehat{\lambda}_j|^2 - |\mu_j|^2 = 0$, so that Eq. (3) gives

$$|t^{1/\alpha} + \widehat{\lambda}_j|^2 - \frac{|\mu_j|^2}{|\widehat{\lambda}_j|^2} t^{2/\alpha} = 2\operatorname{Re}(\widehat{\lambda}_j)t^{1/\alpha} + |\widehat{\lambda}_j|^2 \geq 0. \quad (4)$$

If $\operatorname{Re}(\widehat{\lambda}_j) < 0$, then $|\widehat{\lambda}_j|^2 - |\mu_j|^2 = \operatorname{Re}(\widehat{\lambda}_j)^2$, so that Eq. (3) yields

$$|t^{1/\alpha} + \widehat{\lambda}_j|^2 - \frac{|\mu_j|^2}{|\widehat{\lambda}_j|^2} t^{2/\alpha} = \frac{\operatorname{Re}(\widehat{\lambda}_j)^2}{|\widehat{\lambda}_j|^2} t^{2/\alpha} + 2\operatorname{Re}(\widehat{\lambda}_j)t^{1/\alpha} + |\widehat{\lambda}_j|^2 = \left(\frac{\operatorname{Re}(\widehat{\lambda}_j)}{|\widehat{\lambda}_j|} t^{1/\alpha} + |\widehat{\lambda}_j| \right)^2 \geq 0. \quad (5)$$

From Eqs. (4) and (5), it holds that $|t^{1/\alpha} + \widehat{\lambda}_j|^2 - (|\mu_j|^2/|\widehat{\lambda}_j|^2)t^{2/\alpha} \geq 0$, which shows $|t^{1/\alpha} + \widehat{\lambda}_j| \geq (|\mu_j|/|\widehat{\lambda}_j|)t^{1/\alpha}$. This and $|t^{1/\alpha} + \widehat{\lambda}_j| \geq |\mu_j|$ give

$$\int_0^{\infty} \frac{t^{1/\alpha}}{|t^{1/\alpha} + \widehat{\lambda}_j|^k} dt = \int_0^x \frac{t^{1/\alpha}}{|t^{1/\alpha} + \widehat{\lambda}_j|^k} dt + \int_x^{\infty} \frac{t^{1/\alpha}}{|t^{1/\alpha} + \widehat{\lambda}_j|^k} dt \leq \int_0^x \frac{t^{1/\alpha}}{|\mu_j|^k} dt + \int_x^{\infty} \frac{t^{1/\alpha}}{(|\mu_j|/|\widehat{\lambda}_j|)t^{1/\alpha})^k} dt = \varphi_k(x). \quad \square$$

The minimum value $\min_{x \in (0, \infty)} \varphi_k(x)$ uniquely exists, and it holds that

$$\min_{x \in (0, \infty)} \varphi_k(x) = \varphi_k(\widehat{\lambda}_j^\alpha) = \frac{k\alpha |\widehat{\lambda}_j|^{\alpha+1}}{|\mu_j|^k (\alpha + 1)(k - \alpha - 1)} =: \zeta_k. \quad (6)$$

Hence, $\int_0^\infty t^{1/\alpha} / |t^{1/\alpha} + \widehat{\lambda}_j|^k dt$ will be estimated on the basis of [Corollary 3](#).

Corollary 3. Let α , $\widehat{\lambda}_j$, and μ_j be as in [Lemma 4](#) and ζ_k be as in [Eq. \(6\)](#). If $|\mu_j| > 0$, then $\int_0^\infty t^{1/\alpha} / |t^{1/\alpha} + \widehat{\lambda}_j|^k dt \leq \zeta_k$.

3. NSD-based algorithm

The algorithm is developed in three steps. Section [3.1](#) introduces the VBD based on NSD. Section [3.2](#) establishes the computable perturbation bound for enclosing the real powers of diagonal blocks. Section [3.3](#) proposes the overall algorithm.

3.1. The VBD based on NSD

Assume as the result of the NSD of A , the matrices $\Lambda, X \in \mathbb{C}^{n \times n}$ with $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$ are obtained, such that $AX \approx X\Lambda$. By executing column permutation, if necessary, let $\{\lambda_{i_1^{(j)}}, \dots, \lambda_{i_{p_j}^{(j)}}\}$, $j = 1, \dots, q$ be sets of clusters, where the positive integers $i_1^{(1)}, \dots, i_{p_1}^{(1)}, \dots, i_1^{(q)}, \dots, i_{p_q}^{(q)}$ satisfy $1 = i_1^{(1)} < \dots < i_{p_1}^{(1)} < \dots < i_1^{(q)} < \dots < i_{p_q}^{(q)} = n$ and $p_1 + \dots + p_q = n$. Note that the case where λ_j is isolated from the others is included in the $p_j = 1$ case. In [\[23, Algorithm 1\]](#), first the nonnegative n -vector r such that $\sigma(A) \subseteq \bigcup_{i=1}^n \langle \lambda_i, r_i \rangle$ is computed. Then, the cluster $\{\lambda_{i_1^{(j)}}, \dots, \lambda_{i_{p_j}^{(j)}}\}$ is identified by checking whether $\bigcup_{k=1}^{p_j} \langle \lambda_{i_k^{(j)}}, r_{i_k^{(j)}} \rangle$ is a connected domain isolated from other intervals. Also, let $X_j := [X_{:i_1^{(j)}}, \dots, X_{:i_{p_j}^{(j)}}]$ for $j = 1, \dots, q$, and $W_j \in \mathbb{C}^{n \times p_j}$ and $P_j \in \mathbb{C}^{p_j \times p_j}$ satisfy $AW_j = W_j P_j$ and $\bigcup_{j=1}^q \sigma(P_j) = \sigma(A)$. If [\[23, Algorithm 1\]](#) terminates successfully, then it gives $\Delta_j \in \mathbb{R}_+^{n \times p_j}$, $\widehat{\lambda}_j \in \mathbb{C}$, and $R_j \in \mathbb{R}_+^{p_j \times p_j}$ such that $\langle X_j, \Delta_j \rangle \ni W_j$ and $\langle \widehat{\lambda}_j I_{p_j}, R_j \rangle \ni P_j$ with X_j and $\lambda_{i_1^{(j)}}, \dots, \lambda_{i_{p_j}^{(j)}}$ being inputs, for $j = 1, \dots, q$. Note that there is a case when [\[23, Algorithm 1\]](#) can fail if X is singular or ill-conditioned. Let $W := [W_1, \dots, W_q] \in \mathbb{C}^{n \times n}$, $\Delta := [\Delta_1, \dots, \Delta_q] \in \mathbb{R}_+^{n \times n}$, and $\mathbf{W} := (X, \Delta) \in \mathbb{I}\mathbb{C}^{n \times n}$. Then, $W \in \mathbf{W}$ and

$$AW = [AW_1, \dots, AW_q] = [W_1 P_1, \dots, W_q P_q] = W \text{diag}(P_1, \dots, P_q).$$

Nonsingularity of any matrix contained in $\mathbf{W}$ can be verified by executing a known algorithm (for instance, the INTLAB routine `verifylss`). If the verification succeeds, then W is also nonsingular, so that $A = W \text{diag}(P_1, \dots, P_q) W^{-1}$. Thus, $\mathbf{W}$ and $\text{diag}(\langle \widehat{\lambda}_1 I_{p_1}, R_1 \rangle, \dots, \langle \widehat{\lambda}_q I_{p_q}, R_q \rangle)$ are the results of VBD. VBD is also utilized in [\[19\]](#). In our implementation, we compute enclosures for dot products in $AX - X\Lambda$ with quadruple precision (see [\[25\]](#), for instance) to make the components of Δ_j and R_j small. The VBD in Section [4](#) is implemented similarly. From [Lemma 1](#), we have $A^\alpha = W \text{diag}(P_1^\alpha, \dots, P_q^\alpha) W^{-1}$, so that the problem of enclosing A^α can be reduced to that of enclosing $P_1^\alpha, \dots, P_q^\alpha$.

3.2. Computable perturbation bound

For $j = 1, \dots, q$, the matrix P_j can be written as $P_j = \widehat{\lambda}_j I_{p_j} + Q_j$, where $Q_j \in \mathbb{C}^{p_j \times p_j}$ satisfies $|Q_j| \leq R_j$. If $p_j = 1$, then P_j^α can be enclosed by executing a known algorithm with $\langle \widehat{\lambda}_j, R_j \rangle$ and α being inputs. In our implementation, we enclose P_j^α when $p_j = 1$ via the INTLAB functions `exp` and `log`. On the other hand, if $p_j \geq 2$, then this approach is not possible. In order to enclose P_j^α when $p_j \geq 2$, we formulate and prove [Theorem 1](#), which gives an upper bound for $|\langle \widehat{\lambda}_j I_{p_j} + Q_j \rangle^\alpha - \widehat{\lambda}_j^\alpha I_{p_j}|$ when $\alpha \in (0, 1)$.

Theorem 1. Let $\alpha \in (0, 1)$, $\widehat{\lambda}_j \in \mathbb{C}$, $Q_j \in \mathbb{C}^{p_j \times p_j}$, $R_j \in \mathbb{R}_+^{p_j \times p_j}$, $v := R_j \mathbb{1}_{p_j}$, and μ_j and ζ_k be as in [Lemma 4](#) and [Eq. \(6\)](#), respectively. Suppose $|Q_j| \leq R_j$ and $\|v\|_\infty < |\mu_j|$, and define $w := [1/(|\mu_j| - v_1), \dots, 1/(|\mu_j| - v_{p_j})]^T$. Then, $\sigma(\widehat{\lambda}_j I_{p_j}) \cap \mathbb{R}_- = \emptyset$, $\sigma(\widehat{\lambda}_j I_{p_j} + Q_j) \cap \mathbb{R}_- = \emptyset$, and

$$|\langle \widehat{\lambda}_j I_{p_j} + Q_j \rangle^\alpha - \widehat{\lambda}_j^\alpha I_{p_j}| \leq S_j, \quad \text{where } S_j := \frac{\sin(\alpha\pi) \zeta_2}{\alpha\pi} R_j (I_{p_j} + v w^T).$$

Remark 3. It follows from $\alpha \in (0, 1)$ that $\alpha\pi > 0$ and $\sin(\alpha\pi) > 0$.

Proof. Let $t \in [0, \infty)$. From $|t^{1/\alpha} + \widehat{\lambda}_j| \geq |\mu_j| > \|v\|_\infty \geq 0$, $\forall t$, the matrix $(t^{1/\alpha} + \widehat{\lambda}_j) I_{p_j}$ is nonsingular for any t . This and [Lemma 2](#) show $\sigma(\widehat{\lambda}_j I_{p_j}) \cap \mathbb{R}_- = \emptyset$.

It holds that $\| |Q_j| \mathbb{I}_{p_j} \|_\infty \leq \| R_j \mathbb{I}_{p_j} \|_\infty = \| v \|_\infty < |\mu_j| \leq |t^{1/\alpha} + \widehat{\lambda}_j|, \forall t$. Let

$$\widehat{w}(t) := \left[\frac{1}{|t^{1/\alpha} + \widehat{\lambda}_j| - (|Q_j| \mathbb{I}_{p_j})_1}, \dots, \frac{1}{|t^{1/\alpha} + \widehat{\lambda}_j| - (|Q_j| \mathbb{I}_{p_j})_{p_j}} \right]^T.$$

It then follows that $\widehat{w}(t) \leq w, \forall t$. From this, $\| |Q_j| \mathbb{I}_{p_j} \|_\infty < |t^{1/\alpha} + \widehat{\lambda}_j|, \forall t$ and [Corollary 2](#) applied to $\beta := t^{1/\alpha} + \widehat{\lambda}_j$ and $S := Q_j$, the matrix $(t^{1/\alpha} + \widehat{\lambda}_j)I_{p_j} + Q_j$ is nonsingular for any t , and

$$|((t^{1/\alpha} + \widehat{\lambda}_j)I_{p_j} + Q_j)^{-1}| \leq \frac{1}{|t^{1/\alpha} + \widehat{\lambda}_j|} (I_{p_j} + |Q_j| \mathbb{I}_{p_j} \widehat{w}(t)^T) \leq \frac{1}{|t^{1/\alpha} + \widehat{\lambda}_j|} (I_{p_j} + v w^T), \quad \forall t. \quad (7)$$

The nonsingularity of $(t^{1/\alpha} + \widehat{\lambda}_j)I_{p_j} + Q_j$ for any t and [Lemma 2](#) yield $\sigma(\widehat{\lambda}_j I_{p_j} + Q_j) \cap \mathbb{R}_- = \emptyset$.

For $M, N \in \mathbb{C}^{n \times n}$, we have $(M + N)^{-1} = M^{-1}(I_n - N(M + N)^{-1})$ if M and $M + N$ are nonsingular. Therefore,

$$((t^{1/\alpha} + \widehat{\lambda}_j)I_{p_j} + Q_j)^{-1} = \frac{1}{t^{1/\alpha} + \widehat{\lambda}_j} (I_{p_j} - Q_j((t^{1/\alpha} + \widehat{\lambda}_j)I_{p_j} + Q_j)^{-1}).$$

This equality, $\alpha \in (0, 1)$, $\sigma(\widehat{\lambda}_j I_{p_j}) \cap \mathbb{R}_- = \emptyset$, $\sigma(\widehat{\lambda}_j I_{p_j} + Q_j) \cap \mathbb{R}_- = \emptyset$ and [Eq. \(1\)](#) yield

$$\begin{aligned} (\widehat{\lambda}_j I_{p_j} + Q_j)^\alpha &= \frac{\sin(\alpha\pi)}{\alpha\pi} (\widehat{\lambda}_j I_{p_j} + Q_j) \int_0^\infty ((t^{1/\alpha} + \widehat{\lambda}_j)I_{p_j} + Q_j)^{-1} dt \\ &= \frac{\sin(\alpha\pi)}{\alpha\pi} \int_0^\infty \left(\frac{\widehat{\lambda}_j}{t^{1/\alpha} + \widehat{\lambda}_j} (I_{p_j} - Q_j((t^{1/\alpha} + \widehat{\lambda}_j)I_{p_j} + Q_j)^{-1}) + Q_j((t^{1/\alpha} + \widehat{\lambda}_j)I_{p_j} + Q_j)^{-1} \right) dt \\ &= \widehat{\lambda}_j^\alpha I_{p_j} + \frac{\sin(\alpha\pi)}{\alpha\pi} \int_0^\infty \frac{t^{1/\alpha}}{t^{1/\alpha} + \widehat{\lambda}_j} Q_j((t^{1/\alpha} + \widehat{\lambda}_j)I_{p_j} + Q_j)^{-1} dt, \end{aligned}$$

so that $|(\widehat{\lambda}_j I_{p_j} + Q_j)^\alpha - \widehat{\lambda}_j^\alpha I_{p_j}| \leq \widehat{S}_j$, where

$$\widehat{S}_j := \frac{\sin(\alpha\pi)}{\alpha\pi} \int_0^\infty \frac{t^{1/\alpha}}{|t^{1/\alpha} + \widehat{\lambda}_j|} |Q_j| |((t^{1/\alpha} + \widehat{\lambda}_j)I_{p_j} + Q_j)^{-1}| dt.$$

The inequalities [\(7\)](#) and $|Q_j| \leq R_j$, and [Corollary 3](#) prove $\widehat{S}_j \leq S_j$. $\square$

Remark 4. [Theorem 1](#) is also valid when $p_j = 1$. Therefore, $\sigma(\widehat{\lambda}_j I_{p_j} + Q_j) \cap \mathbb{R}_- = \emptyset$ can be verified by checking $\|v\|_\infty < |\mu_j|$ even when $p_j = 1$.

Remark 5. If $\sigma(\widehat{\lambda}_j I_{p_j} + Q_j) \cap \mathbb{R}_- = \emptyset, \forall j$, then $\sigma(A) \cap \mathbb{R}_- = \emptyset$ because $\widehat{\lambda}_j I_{p_j} + Q_j = P_j$ and $\sigma(A) = \bigcup_{j=1}^q \sigma(P_j)$. [Algorithms 2](#) and [3](#) prove $\sigma(A) \cap \mathbb{R}_- = \emptyset$ in this manner.

From [Theorem 1](#), it follows that $P_j^\alpha \in (\widehat{\lambda}_j^\alpha I_{p_j}, S_j)$ if $\alpha \in (0, 1)$. In practical execution, α may be an interval α to represent irrational numbers. The case $\alpha \in \mathbb{R}$ is included in the case $\alpha \in \mathbb{IR}$. Thus, an algorithm for enclosing P_j^α in the latter case is developed. In [Algorithms 1](#) to [4](#), all interval computations are performed in machine interval arithmetic.

Algorithm 1. Let $P_j, p_j, R_j, \mu_j, \widehat{\lambda}_j, v, S_j$, and α be as above, and $\bar{S}_j \in \mathbb{R}_+^{p_j \times p_j}$ satisfy $\bar{S}_j \geq S_j, \forall \alpha \in \alpha$. This algorithm computes $P_j^\alpha \in \mathbb{IC}^{p_j \times p_j}$ such that $P_j^\alpha \supseteq \{P_j^\alpha : \alpha \in \alpha\}$ when $\widehat{\lambda}_j, R_j$, and α are given.

Step 1. If $\|v\|_\infty < |\mu_j|$ cannot be verified, then terminate with failure.

Step 2. If $\inf(\alpha) = \sup(\alpha) = 0$, then let $P_j^\alpha = I_{p_j}$ and terminate.

Step 3. If $p_j = 1$, then compute P_j^α via a known algorithm (for instance, combination of the INTLAB functions `exp` and `log`) and terminate.

Step 4. If $\text{diam}(\alpha) < 1$ cannot be verified, then terminate with failure.

Step 5. If $\alpha \subseteq (0, 1)$, then compute P_j^α by $P_j^\alpha = (\widehat{\lambda}_j I_{p_j})^\alpha + \langle 0, \bar{S}_j \rangle$ and terminate.

Step 6. Let $\xi := \lfloor \text{mid}(\alpha) \rfloor$ and $\delta := \alpha - \xi$. If $\inf(\delta) = \sup(\delta) = 0$, then let $\mathbf{P}_j^\delta = I_{p_j}$. If $\delta \subseteq (0, 1)$, then compute $\mathbf{P}_j^\delta \in \mathbb{IC}^{p_j \times p_j}$ analogously to Step 5, where α is replaced by δ . If both of them are not true, then terminate with failure.

Step 7. Calculate $\mathbf{P}_j^\xi \in \mathbb{IC}^{p_j \times p_j}$ by $\mathbf{P}_j^\xi = \begin{cases} \langle \hat{\lambda}_j I_{p_j}, R_j \rangle^\xi & (\text{if } \xi > 0) \\ (\langle \hat{\lambda}_j I_{p_j}, R_j \rangle^{-1})^{|\xi|} & (\text{if } \xi < 0) \end{cases}$. Compute $\mathbf{P}_j^\alpha = \mathbf{P}_j^\xi \mathbf{P}_j^\delta$ and terminate.

In Step 6, the case $\inf(\delta) = \sup(\delta) = 0$ occurs when $\inf(\alpha)$ and $\sup(\alpha)$ are the same nonzero integers. In Step 7, the case $\xi = 0$ does not occur because Algorithm 1 terminates before Step 7 if $\xi = 0$. In Step 7, we execute [2, Algorithm 4.1] with $\langle \hat{\lambda}_j I_{p_j}, R_j \rangle$ or $\langle \hat{\lambda}_j I_{p_j}, R_j \rangle^{-1}$, and $|\xi|$ being the inputs. In this execution, machine interval arithmetic for interval matrices is carried out instead of floating point arithmetic for point matrices. Observe that any matrix contained in $\langle \hat{\lambda}_j I_{p_j}, R_j \rangle$ is nonsingular, which follows from $\sigma(\hat{\lambda}_j I_{p_j} + Q_j) \cap \mathbb{R}_- = \emptyset$. Moreover, $\langle \hat{\lambda}_j I_{p_j}, R_j \rangle^{-1}$ can be obtained by executing a known algorithm. In our implementation, `verifylss` is adopted. The computational cost of Algorithm 1 is $\mathcal{O}(p_j^2)$ if $\xi = 0$; otherwise, the cost is $\mathcal{O}(\max(1, \lfloor \log_2 |\xi| \rfloor) p_j^3)$ (see [2, Algorithm 4.1]).

3.3. Overall algorithm

On the basis of Sections 3.1 and 3.2, an algorithm for enclosing A^α is proposed.

Algorithm 2. Let P_j and $\mathbf{W}$ be as in Section 3.1, $\mathbf{P}_j^\alpha$ be as in Algorithm 1, and $\mathbf{P}_j \in \mathbb{IC}^{p_j \times p_j}$ contain P_j for $j = 1, \dots, q$. This algorithm computes $\mathbf{A}^\alpha \in \mathbb{IC}^{n \times n}$ such that $\mathbf{A}^\alpha \supseteq \{A^\alpha : \alpha \in \alpha\}$. If the algorithm terminates successfully, then $\sigma(A) \cap \mathbb{R}_- = \emptyset$ is moreover proven.

Step 1. Compute $\mathbf{W}$ and $\mathbf{P}_j, j = 1, \dots, q$ by executing [23, Algorithm 1].

Step 2. Compute $\mathbf{P}_j^\alpha$ for all j by repeatedly executing Algorithm 1. If the computation failed for at least one j , then terminate with failure.

Step 3. Compute $\mathbf{A}^\alpha$ by $\mathbf{A}^\alpha = \mathbf{W} \text{diag}(\mathbf{P}_1^\alpha, \dots, \mathbf{P}_q^\alpha) \mathbf{W}^{-1}$ and terminate.

Step 1 involves $\mathcal{O}(n^3)$ operations (see [23, Section 3.4]). In our implementation, the interval matrix $\mathbf{W}^{-1}$ is computed via `verifylss`. Then, Step 3 also requires $\mathcal{O}(n^3)$ operations. From $\sum_{j=1}^q p_j = n$, Step 2 requires $\mathcal{O}(n^2)$ operations ($\mathcal{O}(\max(1, \lfloor \log_2 |\xi| \rfloor) n^3)$ operations) if $\xi = 0$ (if $\xi \neq 0$), where $\xi := \lfloor \text{mid}(\alpha) \rfloor$. Thus, Algorithm 2 involves $\mathcal{O}(n^3)$ operations if $\xi = 0$, or $\xi \neq 0$ and $\lfloor \log_2 |\xi| \rfloor$ is $\mathcal{O}(1)$.

4. Algorithm based on the NJD

Let N_j and n_j be as in Section 2. When A is defective or close to defective, the matrix X in Section 3.1 becomes singular or ill-conditioned, causing failure to [23, Algorithm 1]. In such situations, the NJD $AZ \approx ZJ$ can be utilized, where $Z, J \in \mathbb{C}^{n \times n}$, Z is nonsingular, $J = \text{diag}(J_1, \dots, J_p)$, $J_j = \lambda_j I_{n_j} + N_j, j = 1, \dots, p$, and $\sum_{j=1}^p n_j = n$.

The following sentences in [18] are cited for explaining the NJD in [26].

Computing a Jordan canonical form of A is ill-conditioned, since the Jordan form is wildly sensitive to perturbation. Any kind of error completely changes the form. Differently from a Jordan decomposition, the NJD in [26] computes a Jordan form of an approximation of A in order to reduce the ill-conditioning. To explicitly illustrate the two decompositions and explain how the NJD helps, we give an example. Let $A = \begin{bmatrix} 1 & 1 \\ 0 & 1 + 10^{-10} \end{bmatrix}$. In this case, the Jordan decomposition of A coincides with the diagonalization of A , and $A = Z_* J_* Z_*^{-1}$, where $J_* := \text{diag}(1, 1 + 10^{-10})$ and $Z_* := \begin{bmatrix} 1 & 1 \\ 0 & 10^{-10} \end{bmatrix}$. The MATLAB routine `cond` gives $\kappa(Z_*) \approx 2.0\text{e}+10$. As the result of the NJD, on the other hand, we obtain $A \approx \tilde{Z} \tilde{J} \tilde{Z}^{-1}$, where

$$\tilde{J} := \begin{bmatrix} 1 + 5.0 \times 10^{-11} & 1 \\ 0 & 1 + 5.0 \times 10^{-11} \end{bmatrix}, \quad \tilde{Z} := \begin{bmatrix} -1 & 5.0000015 \times 10^{-11} \\ -5.0000015 \times 10^{-11} & -1 \end{bmatrix}.$$

Computations by MATLAB yield $\|A\tilde{Z} - \tilde{Z}\tilde{J}\|_2 \approx 2.5\text{e}-21$ and $\kappa(\tilde{Z}) \approx 1.0\text{e}+0$. We see $\kappa(\tilde{Z}) \ll \kappa(Z_*)$, although the Jordan structures of J_* and $\tilde{J}$ are different.

We continue our discussion in a similar manner as in Section 3.

4.1. The VBD based on the NJD

Let $q, p_j, i_1^{(j)}, \dots, i_{p_j}^{(j)}, W_j, P_j$, and $\mathbf{W}$ be as in Section 3.1. We can execute [23, Algorithm 3] by utilizing the NJD instead of the numerical block diagonalization in [23, Section 4]. Then, $\hat{\lambda}_j \in \mathbb{C}$, $R_j \in \mathbb{R}_+^{p_j \times p_j}$, $X_j \in \mathbb{C}^{n \times p_j}$, and $\Delta_j \in \mathbb{R}_+^{n \times p_j}$ can be obtained such that $(X_j, \Delta_j) \ni W_j$, $(\hat{\lambda}_j I_{p_j} + M_j, R_j) \ni P_j$, and $\bigcup_{j=1}^q \sigma(P_j) = \sigma(A)$, where $M_j = \text{diag}(N_{k_1^{(j)}}, \dots, N_{k_{s_j}^{(j)}})$ and $\sum_{\ell=1}^{s_j} n_{k_\ell^{(j)}} = p_j$. If verification for nonsingularity of any matrix contained in $\mathbf{W}$ is succeeded, then $\mathbf{W}$ and $\text{diag}((\hat{\lambda}_1 I_{p_1} + M_1, R_1), \dots, (\hat{\lambda}_q I_{p_q} + M_q, R_q))$ are the results of VBD, which are also mentioned in [19]. Unlike the VBD in Section 3.1, the cluster indices $i_1^{(j)}, \dots, i_{p_j}^{(j)}$ are identified by checking $|\lambda_j - \lambda_k| \leq \eta$ for $k \neq j$ with η being a tolerance. In our implementation, we set $\eta = \sqrt{\text{eps}}$, where eps is machine epsilon.

4.2. Computable perturbation bound

The diagonal block P_j can be written as $P_j = \hat{\lambda}_j I_{p_j} + M_j + Q_j$, where $Q_j \in \mathbb{C}^{p_j \times p_j}$ satisfies $|Q_j| \leq R_j$. An upper bound for $|(\hat{\lambda}_j I_{p_j} + M_j + Q_j)^\alpha - (\hat{\lambda}_j I_{p_j} + M_j)^\alpha|$ is derived to enclose P_j^α when $p_j \geq 2$ and $\alpha \in (0, 1)$.

Theorem 2. Let $n_{k_\ell^{(j)}}$, $\ell = 1, \dots, s_j$ and M_j be as above, μ_j and ζ_k be as in Lemma 4 and Eq. (6), respectively, $\hat{\lambda}_j$, Q_j , R_j , and v be as in Theorem 1, $\alpha \in (0, 1)$, $n_{\max} := \max_\ell n_{k_\ell^{(j)}}$, $U := \sum_{i=0}^{n_{\max}-1} (1/|\mu_j|^i) M_j^i$, and $u := Uv$. Suppose $|Q_j| \leq R_j$ and $\|u\|_\infty < |\mu_j|$, and define

$$w := \left[\max_i \frac{(U_{:1})_i}{|\mu_j| - u_i}, \dots, \max_i \frac{(U_{:p_j})_i}{|\mu_j| - u_i} \right]^T.$$

Then, $\sigma(\hat{\lambda}_j I_{p_j} + M_j) \cap \mathbb{R}_- = \emptyset$, $\sigma(\hat{\lambda}_j I_{p_j} + M_j + Q_j) \cap \mathbb{R}_- = \emptyset$, and $|(\hat{\lambda}_j I_{p_j} + M_j + Q_j)^\alpha - (\hat{\lambda}_j I_{p_j} + M_j)^\alpha| \leq T_j$, where

$$T_j := \frac{\sin(\alpha\pi)}{\alpha\pi} \left(\sum_{i=0}^{n_{\max}-1} \sum_{k=0}^{n_{\max}-1} \zeta_{i+k+2} M_j^i R_j M_j^k \right) (I_{p_j} + v w^T).$$

Remark 6. When $n_{\max} = 1$, Theorem 2 reduces to Theorem 1. Therefore, Theorem 2 is a generalization of Theorem 1.

Proof. Let $t \in [0, \infty)$ and $Y(t) := (t^{1/\alpha} + \hat{\lambda}_j) I_{p_j} + M_j$. Similar to the proof of Theorem 1, the matrix $Y(t)$ is nonsingular for any t , and $\sigma(\hat{\lambda}_j I_{p_j} + M_j) \cap \mathbb{R}_- = \emptyset$.

From $\rho(M_j) = 0$, the Neumann series, $M_j^{n_{\max}} = 0$ and $|t^{1/\alpha} + \hat{\lambda}_j| \geq |\mu_j|$, $\forall t$, it follows for any t that

$$\begin{aligned} |Y(t)^{-1}| &= \frac{1}{|t^{1/\alpha} + \hat{\lambda}_j|} \left| \left(I_n + \frac{1}{t^{1/\alpha} + \hat{\lambda}_j} M_j \right)^{-1} \right| = \frac{1}{|t^{1/\alpha} + \hat{\lambda}_j|} \left| \sum_{i=0}^{\infty} \left(\frac{-1}{t^{1/\alpha} + \hat{\lambda}_j} \right)^i M_j^i \right| \\ &= \frac{1}{|t^{1/\alpha} + \hat{\lambda}_j|} \left| \sum_{i=0}^{n_{\max}-1} \left(\frac{-1}{t^{1/\alpha} + \hat{\lambda}_j} \right)^i M_j^i \right| \\ &\leq \sum_{i=0}^{n_{\max}-1} \frac{1}{|t^{1/\alpha} + \hat{\lambda}_j|^{i+1}} M_j^i \\ &\leq \frac{1}{|\mu_j|} U. \end{aligned} \tag{8}$$

$$\leq \frac{1}{|\mu_j|} U. \tag{9}$$

Let $y(t) := |Y(t)^{-1}| |Q_j| \mathbb{1}_{p_j}$. The inequalities (9), $|Q_j| \leq R_j$, and $\|u\|_\infty < |\mu_j|$ show

$$\|y(t)\|_\infty \leq \|UR_j \mathbb{1}_{p_j}\|_\infty / |\mu_j| = \|Uv\|_\infty / |\mu_j| = \|u\|_\infty / |\mu_j| < 1, \quad \forall t. \tag{10}$$

Define $\hat{w}(t) := [|Y(t)^{-1}|_{:1} y(t), \dots, |Y(t)^{-1}|_{:p_j} y(t)]^T$. The inequalities (9) and (10) yield

$$\hat{w}(t) \leq [|(U/|\mu_j|)_{:1}| \|u/|\mu_j|\|, \dots, |(U/|\mu_j|)_{:p_j}| \|u/|\mu_j|\|]^T = w, \quad \forall t. \tag{11}$$

The inequalities $|Q_j| \leq R_j$, (10), and (11), and Corollary 1 applied to $S := Q_j$ and $T := Y(t)$ show that $Y(t) + Q_j$ is nonsingular for any t , and

$$|(Y(t) + Q_j)^{-1}| \leq |Y(t)^{-1}|(I_{p_j} + |Q_j| \mathbb{1}_{p_j} \widehat{w}(t)^T) \leq |Y(t)^{-1}|(I_{p_j} + v w^T), \quad \forall t. \quad (12)$$

The nonsingularity of $Y(t) + Q_j$ for any t and Lemma 2 show $\sigma(\widehat{\lambda}_j I_{p_j} + M_j + Q_j) \cap \mathbb{R}_- = \emptyset$.

We have $(Y(t) + Q_j)^{-1} = Y(t)^{-1}(I_{p_j} - Q_j(Y(t) + Q_j)^{-1})$, so that

$$\begin{aligned} (\widehat{\lambda}_j I_{p_j} + M_j + Q_j)^\alpha &= \frac{\sin(\alpha\pi)}{\alpha\pi} (\widehat{\lambda}_j I_{p_j} + M_j + Q_j) \int_0^\infty (Y(t) + Q_j)^{-1} dt \\ &= \frac{\sin(\alpha\pi)}{\alpha\pi} \int_0^\infty ((\widehat{\lambda}_j I_{p_j} + M_j)Y(t)^{-1}(I_{p_j} - Q_j(Y(t) + Q_j)^{-1}) + Q_j(Y(t) + Q_j)^{-1}) dt \\ &= (\widehat{\lambda}_j I_{p_j} + M_j)^\alpha + \frac{\sin(\alpha\pi)}{\alpha\pi} \int_0^\infty t^{1/\alpha} Y(t)^{-1} Q_j(Y(t) + Q_j)^{-1} dt, \end{aligned}$$

which gives $|(\widehat{\lambda}_j I_{p_j} + M_j + Q_j)^\alpha - (\widehat{\lambda}_j I_{p_j} + M_j)^\alpha| \leq \widehat{T}_j$, where

$$\widehat{T}_j := \frac{\sin(\alpha\pi)}{\alpha\pi} \int_0^\infty t^{1/\alpha} |Y(t)^{-1}| |Q_j| |Y(t) + Q_j|^{-1} dt.$$

The inequalities $|Q_j| \leq R_j$, (8), and (12), and Corollary 3 finally prove

$$\begin{aligned} \widehat{T}_j &\leq \frac{\sin(\alpha\pi)}{\alpha\pi} \int_0^\infty t^{1/\alpha} |Y(t)^{-1}| R_j |Y(t)^{-1}| (I_{p_j} + v w^T) dt \\ &\leq \frac{\sin(\alpha\pi)}{\alpha\pi} \int_0^\infty t^{1/\alpha} \left(\sum_{i=0}^{n_{\max}-1} \frac{1}{|t^{1/\alpha} + \widehat{\lambda}_j|^{i+1}} M_j^i \right) R_j \left(\sum_{k=0}^{n_{\max}-1} \frac{1}{|t^{1/\alpha} + \widehat{\lambda}_j|^{k+1}} M_j^k \right) (I_{p_j} + v w^T) dt \\ &= \frac{\sin(\alpha\pi)}{\alpha\pi} \left(\sum_{i=0}^{n_{\max}-1} \sum_{k=0}^{n_{\max}-1} \int_0^\infty \frac{t^{1/\alpha}}{|t^{1/\alpha} + \widehat{\lambda}_j|^{i+k+2}} dt M_j^i R_j M_j^k \right) (I_{p_j} + v w^T) \\ &\leq T_j. \quad \square \end{aligned}$$

From Theorem 2, it follows that $P_j^\alpha \in \langle (\widehat{\lambda}_j I_{p_j} + M_j)^\alpha, T_j \rangle$ if $\alpha \in (0, 1)$. The interval matrix $(\widehat{\lambda}_j I_{p_j} + M_j)^\alpha$ can be enclosed by executing a known algorithm for computing intervals containing scalar real powers. This is because $\widehat{\lambda}_j I_{p_j} + M_j$ is block diagonal, each block has the Jordan structure, and matrix functions of a Jordan block can be represented by scalar functions (see [2, Definition 1.2]).

4.3. Overall algorithm

On the basis of Sections 4.1 and 4.2, Algorithms 3 and 4 are proposed.

Algorithm 3. Let W , P_j , and A^α be as in Algorithm 2. This algorithm computes A^α . Moreover, $\sigma(A) \cap \mathbb{R}_- = \emptyset$ is proven if successful.

Step 1. Compute W and P_j , $j = 1, \dots, q$ by executing the Jordan variant of [23, Algorithm 3].

Step 2. Similar to that in Algorithm 2, in which Algorithm 1 is replaced by Algorithm 4.

Step 3. Similar to that in Algorithm 2.

Algorithm 4. Let v and S_j be as in Theorem 1, P_j^α be as in Algorithm 1, and M_j , $\widehat{\lambda}_j$, u , and T_j be as in Theorem 2. This algorithm computes P_j^α .

Steps 1 to 7. Similar to those in Algorithm 1, where v , $\widehat{\lambda}_j I_{p_j}$, and S_j are replaced by u , $\widehat{\lambda}_j I_{p_j} + M_j$, and T_j , respectively.

The NJD involves $\mathcal{O}(n^4)$ operations. For computing an upper bound for T_j in Theorem 2, it is necessary to execute matrix multiplications $M_j^i R_j M_j^k$, $i, k = 0, \dots, n_{\max} - 1$. On the other hand, M_j is very sparse, so that the computational costs of these multiplications are negligible. Thus, Algorithm 3 involves $\mathcal{O}(n^4)$ operations if $\xi = 0$, or $\xi \neq 0$ and $\lfloor \log_2 |\xi| \rfloor$ is $\mathcal{O}(n)$, where $\xi := \lfloor \text{mid}(\alpha) \rfloor$.

5. Numerical results

A computer with an Intel Core i5-11350H CPU, 16.0 GB RAM and MATLAB R2012a with the Intel Math Kernel Library and IEEE 754 double precision was used. The MATLAB toolbox INTLAB was used for executing machine interval arithmetic operations with circular complex intervals. The compared algorithms are denoted as follows:

Table 1

The RR and CPU times (s) for the “frank” matrix. The upper and lower halves are those when alpha was obtained by Eqs. (13) and (14), respectively.

n	LEs	LEj	RPj	RPj	V
	RR time	RR time	RR time	RR time	RR time
5	8.1e-12	4.5e-12	3.4e-13	8.5e-14	3.5e-14
	7.9e-2	1.1e-1	2.4e-2	3.0e-2	4.0e-2
7	6.7e-8	7.1e-9	2.2e-9	1.3e-10	2.1e-11
	9.3e-2	1.4e-1	3.1e-2	3.7e-2	6.1e-2
9	1.7e-4	3.2e-4	2.4e-6	1.4e-6	1.1e-7
	9.9e-2	2.0e-1	3.4e-2	5.1e-2	9.5e-2
11	6.7e+15	Failed	1.2e+0	7.1e-2	1.1e-3
	1.2e-1	Failed	4.5e-2	7.8e-2	1.1e-1
5	3.1e-12	2.3e-12	1.6e-12	8.3e-14	7.6e-14
	7.7e-2	1.0e-1	2.3e-2	2.9e-2	3.9e-2
7	8.6e-10	6.5e-10	5.3e-10	1.4e-12	4.5e-12
	8.6e-2	1.4e-1	2.7e-2	3.6e-2	6.1e-2
9	1.2e-7	1.2e-6	5.9e-8	4.9e-11	6.4e-10
	9.7e-2	1.8e-1	3.3e-2	5.3e-2	8.2e-2
11	9.5e+15	Failed	4.7e-5	3.0e-8	1.3e-7
	1.2e-1	Failed	4.5e-2	6.7e-2	1.1e-1

LEs: combination of [17, Algorithm 1] and the algorithm in [16, Section 3], where $\sigma(A) \cap \mathbb{R}_- = \emptyset$ is verified,

LEj: combination of [17, Algorithm 2] and the algorithm in [16, Section 4], where $\sigma(A) \cap \mathbb{R}_- = \emptyset$ is verified,

RPj: Algorithm 2, where $\sigma(A) \cap \mathbb{R}_- = \emptyset$ is verified,

RPj: Algorithm 3, where $\sigma(A) \cap \mathbb{R}_- = \emptyset$ is verified, and

V: VERSOFT routine VERMATFUN, where $\sigma(A) \cap \mathbb{R}_- = \emptyset$ is not verified.

In RPs and RPj, the NSD and NJD were performed by MATLAB and NAClab [26] routines eig and NumericalJordanForm, respectively. The routine NumericalJordanForm generally returns not Z and J but $\widehat{Z}$ and $\widehat{J}$ such that $\widehat{AZ} \approx \widehat{ZJ}$, $\widehat{J} = \text{diag}(\widehat{J}_1, \dots, \widehat{J}_p)$, and superdiagonal entries of $\widehat{J}_k$ are not necessarily one. However, Z and J can be computed from $\widehat{Z}$ and $\widehat{J}$ (see [16, Section 5]). In RPs and RPj, [2, Algorithm 4.1] was executed from the Matrix Function Toolbox [27] routine power_binary. In power_binary, by the aid of the INTLAB, machine interval arithmetic was executed instead of floating point arithmetic. See <http://web.cc.iwate-u.ac.jp/~miyajima/MRP.zip> for the details of the implementations, where INTLAB codes for LEs, LEj, RPs, and RPj (denoted by LEs.m, LEj.m, RPs.m, and RPj.m) are uploaded.

We use two different choices for the parameter α , namely interval enclosures for π and $-\pi$ obtained by one of the following INTLAB codes:

$$\text{alpha} = \text{intval}(\pi); \quad (13)$$

$$\text{alpha} = -\text{intval}(\pi); \quad (14)$$

Then, alpha is an interval containing π or $-\pi$. Let $\langle B, R \rangle \ni A^\alpha$. The relative radius $\text{RR} := \|R\|_\infty / \|B\|_\infty$ is defined to assess the quality of enclosure. For some problems, LEs, LEj, RPs, or V failed. The reasons for the failures of LEs and RPs are that the nonsingularity of X in Section 3.1 cannot be verified, that of LEj is enclosing the matrix exponential failed due to overflow, and that of V is enclosing all the eigenvalues and eigenvectors of A failed.

Example 1. The algorithms were applied to four classes of matrices, “frank”, “gcdmat”, “minij”, and “Poisson”, available from the MATLAB gallery function, and matrices of various n were chosen for each of the classes. Tables 1 to 4 display the RR and CPU times (s) of the algorithms. The RR by RPs and RPj were smaller than those by LEs and LEj (see Section 1). The RR by RPs were smaller than those by V except Table 1 and the three problems in Table 2. The RR by RPj were smaller than those by V except the five problems in Table 1. The CPU times of RPs were the smallest of all. The CPU times of RPj were the second smallest except the cases of $n = 400$ in Tables 2 and 3. The CPU times of LEj were the largest of all.

Let $q, p_j, \widehat{\lambda}_j, Q_j$, and S_j be as in Section 3, and M_j and T_j be as in Section 4. From $|(\widehat{\lambda}_j I_{p_j} + Q_j)^\alpha - \widehat{\lambda}_j^\alpha I_{p_j}| \leq S_j$ and $|(\widehat{\lambda}_j I_{p_j} + M_j + Q)^\alpha - (\widehat{\lambda}_j I_{p_j} + M_j)^\alpha| \leq T_j$, it follows that $\|(\widehat{\lambda}_j I_{p_j} + Q_j)^\alpha - \widehat{\lambda}_j^\alpha I_{p_j}\|_\infty \leq \|S_j\|_\infty$ and $\|(\widehat{\lambda}_j I_{p_j} + M_j + Q)^\alpha - (\widehat{\lambda}_j I_{p_j} + M_j)^\alpha\|_\infty \leq \|T_j\|_\infty$, respectively. Therefore, $\|S_j\|_\infty$ and $\|T_j\|_\infty$ can be regarded as normwise perturbation bounds. Let $\mathcal{J} := \{j \in \{1, \dots, q\} : p_j \geq 2\}$, and the integers τ and ω satisfy $\|S_\tau\|_\infty = \max_{j \in \mathcal{J}} \|S_j\|_\infty$ and $\|T_\omega\|_\infty = \max_{j \in \mathcal{J}} \|T_j\|_\infty$, respectively. For clarifying that the componentwise perturbation bounds are useful in practice, Table 5 displays the

Table 2

The RR and CPU times (s) for the “gcdmat” matrix. The upper and lower halves are those when alpha was obtained by Eqs. (13) and (14), respectively.

n	LEs	LEj	RPs	RPj	V
	RR time	RR time	RR time	RR time	RR time
100	4.0e-10	4.9e-10	3.0e-12	2.5e-12	1.8e-11
	4.2e+0	1.2e+1	4.9e-1	1.6e+0	1.0e+1
200	3.6e-9	4.1e-9	7.7e-12	7.6e-12	1.0e-10
	2.4e+1	7.1e+1	2.8e+0	9.6e+0	4.6e+1
300	1.2e-8	1.3e-8	2.0e-11	1.9e-11	3.1e-10
	5.4e+1	1.6e+2	7.4e+0	2.7e+1	7.1e+1
400	3.0e-8	3.3e-8	4.0e-11	4.4e-11	9.0e-10
	7.7e+1	2.8e+2	3.6e+1	9.8e+1	1.4e+2
100	9.8e-11	1.1e-10	2.2e-11	1.1e-11	2.8e-11
	4.6e+0	1.4e+1	5.2e-1	1.6e+0	7.3e+0
200	6.9e-10	7.2e-10	5.6e-11	7.8e-12	5.3e-11
	1.9e+1	6.1e+1	2.5e+0	8.9e+0	3.0e+1
300	1.9e-9	1.9e-9	2.1e-10	3.1e-11	1.6e-10
	4.7e+1	1.5e+2	6.4e+0	2.0e+1	7.1e+1
400	4.1e-9	4.3e-9	4.0e-10	1.8e-10	3.6e-10
	8.7e+1	2.9e+2	3.3e+1	8.9e+1	1.5e+2

Table 3

The RR and CPU times (s) for the “minij” matrix. The upper and lower halves are those when alpha was obtained by Eqs. (13) and (14), respectively.

n	LEs	LEj	RPs	RPj	V
	RR time	RR time	RR time	RR time	RR time
100	6.5e-8	6.5e-8	8.4e-10	8.4e-10	1.1e-8
	4.4e+0	1.5e+1	5.7e-1	1.9e+0	9.3e+0
200	1.7e-6	1.7e-6	7.7e-9	8.2e-9	2.1e-7
	2.0e+1	6.0e+1	1.9e+0	6.6e+0	3.0e+1
300	1.3e-5	1.3e-5	4.5e-8	5.4e-8	1.2e-6
	4.2e+1	1.3e+2	6.0e+0	2.0e+1	6.9e+1
400	5.0e-5	5.0e-5	8.0e-8	8.1e-8	4.1e-6
	7.9e+1	2.9e+2	3.4e+1	9.9e+1	1.5e+2
100	1.4e-5	1.4e-5	1.7e-9	1.5e-9	2.7e-7
	4.9e+0	1.3e+1	5.0e-1	1.3e+0	7.2e+0
200	8.9e-4	8.9e-4	1.6e-8	1.8e-8	1.0e-5
	1.7e+1	6.3e+1	2.1e+0	6.7e+0	3.2e+1
300	1.1e-2	1.1e-2	8.6e-8	6.5e-8	8.2e-5
	4.2e+1	1.3e+2	6.0e+0	1.9e+1	7.8e+1
400	6.3e-2	6.3e-2	1.6e-7	1.8e-7	3.7e-4
	8.7e+1	2.9e+2	3.4e+1	9.8e+1	1.4e+2

normwise bounds $\|S_\tau\|_\infty$ and $\|T_\omega\|_\infty$, and componentwise bounds $\max_{i,k}(S_\tau)_{ik}$, $\min_{i,k}(S_\tau)_{ik}$, $\max_{i,k}(T_\omega)_{ik}$, and $\min_{i,k}(T_\omega)_{ik}$ for the “Poisson” matrix. Note that $\mathcal{J} \neq \emptyset$ in both of RPs and RPj for the “Poisson” matrix, so that τ and ω were well-defined. It can be seen that $\min_{i,k}(S_\tau)_{ik} \ll \|S_\tau\|_\infty$ and $\min_{i,k}(T_\omega)_{ik} \ll \|T_\omega\|_\infty$, although $\max_{i,k}(S_\tau)_{ik} \approx \|S_\tau\|_\infty$ and $\max_{i,k}(T_\omega)_{ik} \approx \|T_\omega\|_\infty$.

Example 2. Consider the case where A comes close to being defective. The algorithms were applied to the problem in [28, Experiment 1], in which

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 1 + \varepsilon \end{bmatrix}, \quad \text{where } \varepsilon \geq 0,$$

whose eigenvector matrix becomes increasingly ill-conditioned as $\varepsilon \rightarrow 0$. Table 6 reports quantities similar to those in Tables 1 to 4 with ε varying from 2^0 to 2^{-52} . The table shows that the RR by LEj and RPj stayed about the same, whereas those by the other algorithms increased as ε decreased.

Table 4

The RR and CPU times (s) for the “Poisson” matrix. The upper and lower halves are those when alpha was obtained by Eqs. (13) and (14), respectively.

n	LEs	LEj	RP s	RPj	V
	RR time	RR time	RR time	RR time	RR time
9	7.7e-13	4.1e-12	1.7e-14	1.9e-14	4.9e-1
	1.0e-1	2.6e-1	5.0e-2	5.8e-2	9.4e-2
36	1.6e-10	1.7e-8	3.7e-13	4.2e-13	Failed
	5.4e-1	2.1e+0	2.1e-1	3.3e-1	Failed
81	3.0e-9	2.3e-7	2.2e-12	2.9e-12	Failed
	2.5e+0	1.1e+1	5.8e-1	1.2e+0	Failed
144	2.9e-8	1.2e-7	1.0e-11	9.1e-12	Failed
	8.1e+0	2.7e+1	1.4e+0	3.5e+0	Failed
9	3.4e-13	4.3e-6	3.0e-14	3.6e-14	2.4e-2
	1.0e-1	1.7e-1	5.4e-2	6.4e-2	9.5e-2
36	9.0e-12	3.6e-11	4.4e-13	5.7e-13	Failed
	5.7e-1	1.6e+0	2.3e-1	3.4e-1	Failed
81	6.7e-11	5.6e-10	2.5e-12	4.9e-12	Failed
	2.7e+0	7.8e+0	6.2e-1	1.2e+0	Failed
144	3.1e-10	3.2e-8	9.7e-12	1.2e-11	Failed
	7.8e+0	3.7e+1	1.6e+0	3.6e+0	Failed

Table 5

The normwise bounds $\|S_\tau\|_\infty$ and $\|T_\omega\|_\infty$, and componentwise bounds $\max_{i,k}(S_\tau)_{ik}$, $\min_{i,k}(S_\tau)_{ik}$, $\max_{i,k}(T_\omega)_{ik}$, and $\min_{i,k}(T_\omega)_{ik}$ for the “Poisson” matrix. The upper and lower halves are those when alpha was obtained by Eqs. (13) and (14), respectively.

n	$\ S_\tau\ _\infty$	$\max(S_\tau)_{ik}$	$\min(S_\tau)_{ik}$	$\ T_\omega\ _\infty$	$\max(T_\omega)_{ik}$	$\min(T_\omega)_{ik}$
9	2.0e-16	2.0e-16	1.4e-30	3.9e-17	3.9e-17	1.5e-30
36	2.8e-16	2.8e-16	7.1e-30	7.9e-16	7.9e-16	5.1e-29
81	5.1e-16	5.1e-16	5.6e-29	7.4e-16	7.4e-16	7.6e-28
144	3.4e-15	3.4e-15	5.7e-28	1.1e-15	1.1e-15	8.7e-28
9	1.5e-15	1.5e-15	1.0e-29	1.4e-15	1.4e-15	1.0e-29
36	2.1e-15	2.1e-15	5.3e-29	1.7e-15	1.7e-15	1.8e-28
81	4.2e-15	4.2e-15	2.9e-28	4.6e-15	4.6e-15	8.9e-28
144	5.2e-15	5.2e-15	8.6e-28	5.5e-15	5.5e-15	4.2e-27

Table 6

The RR and CPU times (s) in Example 2. The upper and lower halves are those when alpha was obtained by Eqs. (13) and (14), respectively.

ε	LEs	LEj	RP s	RPj	V
	RR time	RR time	RR time	RR time	RR time
2^0	5.7e-15	9.0e-15	2.8e-15	3.0e-15	2.8e-15
	4.2e-2	6.6e-2	1.3e-2	1.5e-2	1.5e-2
2^{-26}	6.8e-8	1.2e-14	6.5e-8	2.5e-15	5.3e-8
	3.4e-2	1.1e-1	1.3e-2	8.5e-2	1.5e-2
2^{-39}	5.3e-4	8.9e-15	5.0e-4	2.5e-15	4.1e-4
	3.2e-2	7.7e-2	1.3e-2	3.4e-2	1.5e-2
2^{-48}	2.7e-1	1.3e-14	2.6e-1	2.9e-15	2.1e-1
	3.5e-2	7.9e-2	1.3e-2	3.6e-2	1.6e-2
2^{-52}	1.1e+1	8.3e-15	3.2e+1	2.5e-15	Failed
	3.6e-2	7.5e-2	1.8e-2	3.8e-2	Failed
2^0	2.0e-15	4.6e-15	2.3e-15	2.3e-15	1.8e-15
	4.1e-2	5.2e-2	1.4e-2	1.7e-2	1.8e-2
2^{-26}	5.9e-8	1.0e-14	5.6e-8	5.2e-15	5.0e-8
	3.2e-2	7.7e-2	1.3e-2	4.0e-2	1.5e-2
2^{-39}	4.9e-4	6.9e-15	4.6e-4	6.0e-15	4.1e-4
	3.3e-2	8.0e-2	1.3e-2	3.6e-2	1.7e-2
2^{-48}	2.5e-1	7.1e-15	2.3e-1	5.1e-15	2.2e-1
	3.8e-2	8.6e-2	1.3e-2	3.8e-2	1.7e-2
2^{-52}	8.7e+0	1.2e-14	2.7e+1	8.2e-15	Failed
	3.4e-2	8.3e-2	2.1e-2	3.3e-2	Failed

Table 7

The RR and CPU times (s) in [Example 3](#). The upper and lower halves are those when alpha was obtained by Eqs. (13) and (14), respectively.

γ	LEs	LEj	RP _s	RP _j	V
	RR time	RR time	RR time	RR time	RR time
2^0	Failed	2.1e-9	Failed	6.7e-15	Failed
	Failed	1.7e-1	Failed	8.8e-2	Failed
2^1	Failed	9.6e-11	Failed	5.9e-15	Failed
	Failed	1.9e-1	Failed	5.7e-2	Failed
2^2	Failed	1.1e-11	Failed	5.0e-15	Failed
	Failed	2.1e-1	Failed	5.8e-2	Failed
2^3	Failed	8.2e-12	Failed	6.3e-15	Failed
	Failed	2.7e-1	Failed	5.9e-2	Failed
2^4	Failed	4.6e-12	Failed	6.8e-15	Failed
	Failed	2.7e-1	Failed	5.8e-2	Failed
2^0	Failed	3.5e-9	Failed	8.0e-13	Failed
	Failed	1.5e-1	Failed	5.5e-2	Failed
2^1	Failed	1.5e-10	Failed	8.9e-14	Failed
	Failed	1.8e-1	Failed	5.6e-2	Failed
2^2	Failed	1.8e-11	Failed	1.2e-13	Failed
	Failed	2.2e-1	Failed	5.5e-2	Failed
2^3	Failed	1.2e-11	Failed	1.3e-13	Failed
	Failed	2.6e-1	Failed	5.9e-2	Failed
2^4	Failed	7.7e-12	Failed	9.8e-14	Failed
	Failed	2.9e-1	Failed	5.7e-2	Failed

Table 8

The RR and CPU times (s) in [Example 4](#). The upper and lower halves are those when alpha was obtained by Eqs. (13) and (14), respectively.

γ	LEs	LEj	RP _s	RP _j	V
	RR time	RR time	RR time	RR time	RR time
2^0	Failed	1.3e-8	Failed	2.3e-14	Failed
	Failed	3.6e-1	Failed	1.2e-1	Failed
2^1	Failed	6.3e-10	Failed	1.9e-14	Failed
	Failed	4.2e-1	Failed	1.3e-1	Failed
2^2	Failed	1.1e-10	Failed	2.7e-14	Failed
	Failed	5.5e-1	Failed	1.2e-1	Failed
2^3	Failed	6.2e-11	Failed	5.5e-14	Failed
	Failed	7.4e-1	Failed	1.3e-1	Failed
2^4	Failed	4.9e-11	Failed	1.8e-14	Failed
	Failed	7.3e-1	Failed	1.2e-1	Failed
2^0	Failed	4.9e-8	Failed	1.1e-12	Failed
	Failed	3.9e-1	Failed	1.3e-1	Failed
2^1	Failed	3.4e-9	Failed	9.9e-13	Failed
	Failed	4.8e-1	Failed	1.2e-1	Failed
2^2	Failed	1.6e-10	Failed	5.3e-13	Failed
	Failed	6.3e-1	Failed	1.2e-1	Failed
2^3	Failed	1.5e-10	Failed	5.3e-13	Failed
	Failed	7.8e-1	Failed	1.2e-1	Failed
2^4	Failed	7.8e-11	Failed	1.9e-13	Failed
	Failed	7.2e-1	Failed	1.2e-1	Failed

Example 3. Consider the case where A is defective. Let

$$A_0 := \begin{bmatrix} 2 & 2 & 1 & 0 \\ 0 & 1 & 1 & 1 \\ -1 & -1 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}, \text{ whose Jordan canonical form is } \begin{bmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

We set $A = \gamma A_0$ for a parameter $\gamma \in \mathbb{R}$. [Table 7](#) reports quantities similar to those in [Table 6](#) for various γ , showing that LEj and RPj succeeded for all the problems.

Example 4. Consider the case where A is derogatory. Let $v^{(j)} := (I_8)_{jj}$, $j = 1, \dots, 8$, and $P := [v^{(7)}, v^{(5)}, v^{(3)}, v^{(1)}, v^{(8)}, v^{(6)}, v^{(4)}, v^{(2)}] \in \mathbb{R}^{8 \times 8}$. Then, P is orthogonal. We set $A = \gamma P \text{diag}(A_0, A_0) P^T$ using A_0 and γ in Example 3. Table 8 displays quantities similar to those in Table 7, showing the robustness of LEj and RPj.

6. Conclusion

The author has proposed Algorithms 1 to 4 and reported numerical results. For developing these algorithms, Theorems 1 and 2 have been formulated and proven, providing componentwise perturbation bounds and enabling the verification of $\sigma(A) \cap \mathbb{R}_- = \emptyset$. In many cases of Example 1, Algorithms 2 and 3 gave radii smaller than those by the other algorithms, which were also faster than the others. It can be seen from Examples 2 to 4 that Algorithm 3 worked well even when A was defective or close to defective.

A possible extension of this study is as follows. If A is real, so is A^α (see [1]). We can thus make the enclosure narrower. Future work will be to develop an algorithm applicable even when A is singular.

Acknowledgments

The author acknowledges the referees for their valuable comments. This work was partially supported by JSPS KAKENHI (Grant Number: JP16K05270).

References

- [1] N.J. Higham, L. Lin, A Schur-Padé algorithm for fractional powers of a matrix, *SIAM J. Matrix Anal. Appl.* 32 (2011) 1056–1078.
- [2] N.J. Higham, *Functions of Matrices: Theory and Computation*, SIAM Publications, Philadelphia, 2008.
- [3] T. Charitos, P.R. de Waal, L.C. van der Gaag, Computing short-interval transition matrices of a discrete-time Markov chain from partially observed data, *Stat. Med.* 27 (2008) 905–921.
- [4] R.B. Israel, J.S. Rosenthal, J.Z. Wei, Finding generators for Markov chains via empirical transition matrices, with applications to credit ratings, *Math. Finance* 11 (2001) 245–265.
- [5] K. Burrage, N. Hale, D. Kay, An efficient implicit FEM scheme for fractional-in-space reaction–diffusion equations, *SIAM J. Sci. Comput.* 34 (2012) A2145–A2172.
- [6] M. Arioli, D. Lohin, Discrete interpolation norms with applications, *SIAM J. Numer. Anal.* 47 (2009) 2924–2951.
- [7] S. Fiori, Leap-frog-type learning algorithms over the Lie group of unitary matrices, *Neurocomputing* 71 (2008) 2224–2244.
- [8] N.J. Higham, L. Lin, An improved Schur-Padé algorithm for fractional powers of a matrix and their Fréchet derivatives, *SIAM J. Matrix Anal. Appl.* 34 (2013) 1341–1360.
- [9] F. Tatsuoka, T. Sogabe, Y. Miyatake, S.-L. Zhang, A note on computing the matrix fractional power using the double exponential formula, *Trans. JSIAM* 28 (3) (2018) 142–161, (in Japanese).
- [10] P. Bochev, S. Markov, A self-validating numerical method for the matrix exponential, *Computing* 43 (1989) 59–72.
- [11] P. Bochev, Simultaneous self-verified computation of $\exp(A)$ and $\int_0^1 \exp(As)ds$, *Computing* 45 (1990) 183–191.
- [12] A. Frommer, B. Hashemi, Verified computation of square roots of a matrix, *SIAM J. Matrix Anal. Appl.* 31 (2009) 1279–1302.
- [13] A. Frommer, B. Hashemi, T. Sablik, Computing enclosures for the inverse square root and the sign function of a matrix, *Linear Algebra Appl.* 456 (2014) 199–213.
- [14] S. Miyajima, Fast enclosure for a matrix inverse square root, *Linear Algebra Appl.* 467 (2015) 116–135.
- [15] S. Miyajima, Fast verified computation for the matrix principal p th root, *J. Comput. Appl. Math.* 330 (2018) 276–288.
- [16] S. Miyajima, Verified computation of the matrix exponential, *Adv. Comput. Math.* 45 (2019) 137–152.
- [17] S. Miyajima, Verified computation for the matrix principal logarithm, *Linear Algebra Appl.* 569 (2019) 38–61.
- [18] S. Miyajima, Verified computation for the matrix Lambert W function, *Appl. Math. Comput.* 362 (2019) 124555.
- [19] S. Miyajima, Verified computation of matrix gamma function, *Linear Multilinear Algebra*, <https://doi.org/10.1080/03081087.2020.1757602>.
- [20] S. Miyajima, Verified computation for the geometric mean of two matrices, *Jpn J. Ind. Appl. Math.*, <https://doi.org/10.1007/s13160-020-00448-6>.
- [21] J. Rohn, VERSOFT: Verification Software in MATLAB/INTLAB, <http://uivtx.cs.cas.cz/rohn/matlab>.
- [22] S.M. Rump, INTLAB - INTerval LABoratory, in: T. Csendes (Ed.), *Developments in Reliable Computing*, Kluwer Academic Publishers, Dordrecht, 1999, pp. 77–104.
- [23] S. Miyajima, Fast enclosure for all eigenvalues and invariant subspaces in generalized eigenvalue problems, *SIAM J. Matrix Anal. Appl.* 35 (2014) 1205–1225.
- [24] G. Alefeld, J. Herzberger, *Introduction To Interval Computations*, Academic Press, New York, 1983.
- [25] T. Ogita, S.M. Rump, S. Oishi, Accurate sum and dot product, *SIAM J. Sci. Comput.* 26 (6) (2005) 1955–1988.
- [26] Z. Zeng, T.-Y. Li, NACLab: A Matlab toolbox for numerical algebraic computation, *ACM Commun. Comput. Algebra* 47 (2013) 170–173.
- [27] N.J. Higham, *The Matrix Function Toolbox*, <https://www.maths.manchester.ac.uk/higham/mftoolbox/>.
- [28] M. Fasi, N.J. Higham, B. Iannazzo, An algorithm for the matrix Lambert W function, *SIAM J. Matrix Anal. Appl.* 36 (2015) 669–685.